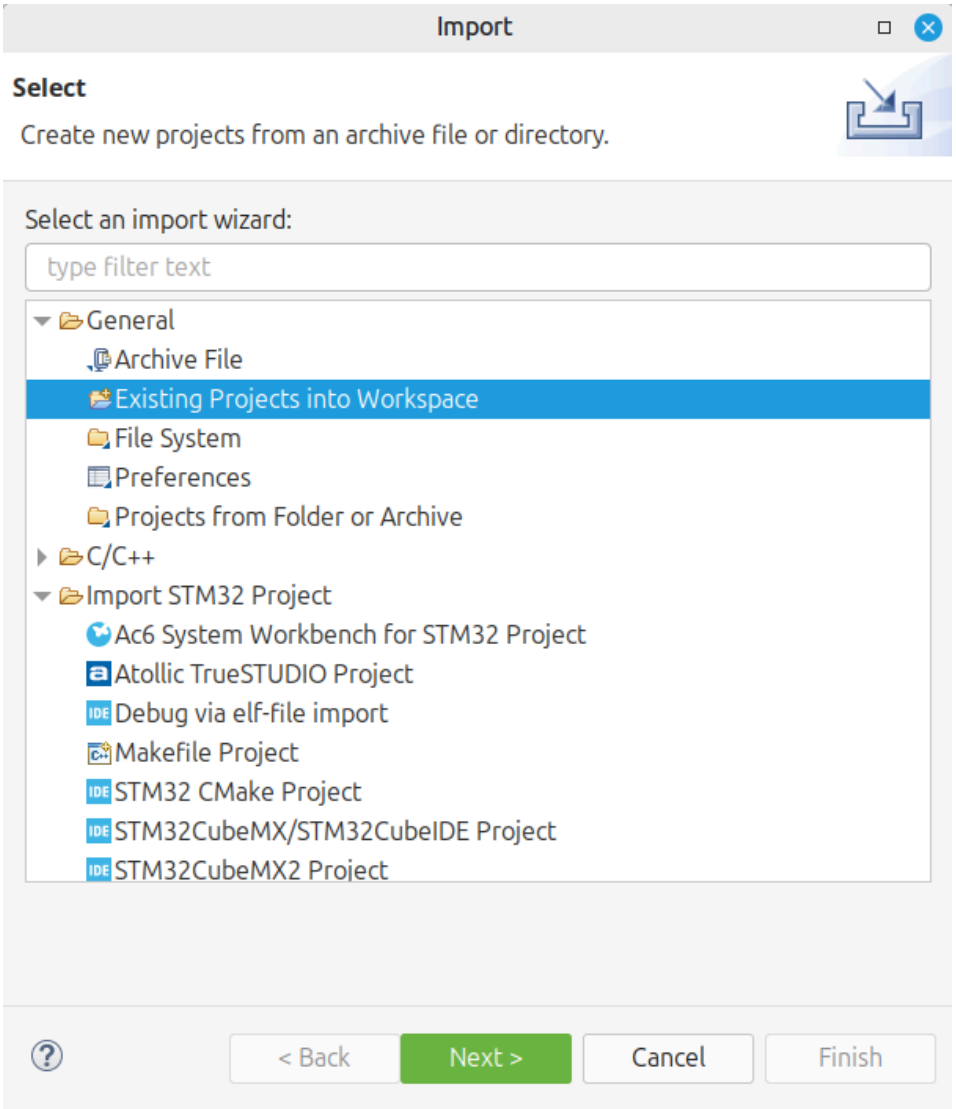
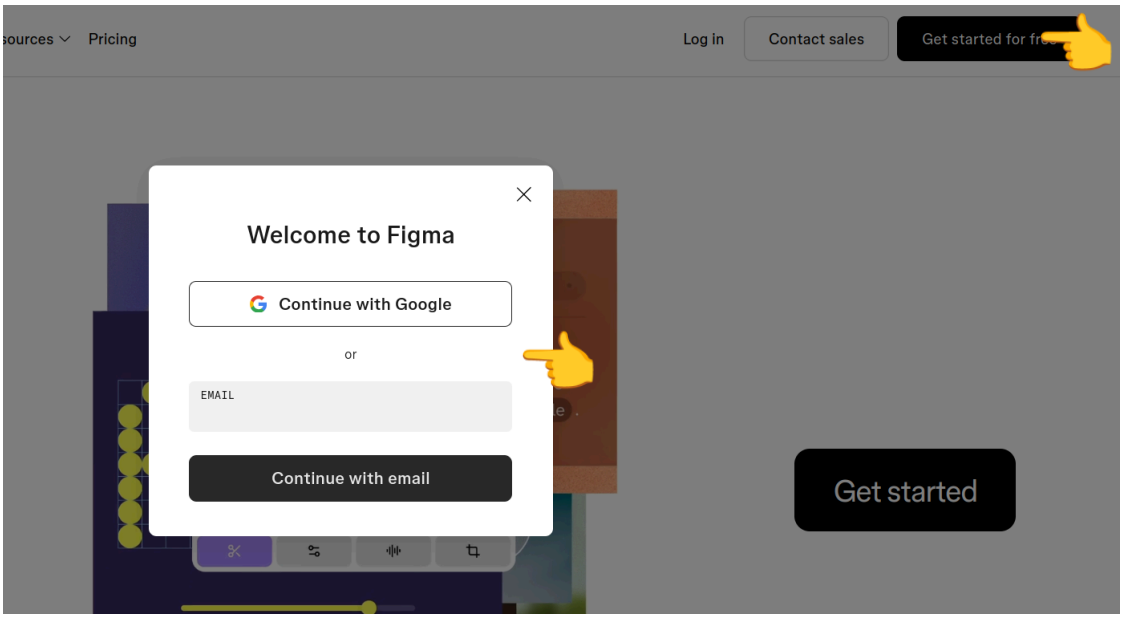
LVGL-Riverdi Guide for Electronica

Prerequisites

1. Download and Install LVGL Pro (currently v2.1.2) from https://lvgl.io/pro
2. Download and Install CubeIDE from https://www.st.com/en/development-tools/stm32cubeide.html
3. Clone the “electronica” branch of this repository into your CubeIDE workspace: `git clone https://github.com/lvgl/lv_port_riverdi_stm32u5 -b electronica --recurse-submodules`
4. Import the project into Cube IDE, starting from **File / Import.** Then select **General / Existing Projects into Workspace.** Close the initial wizard tab if it’s shown.

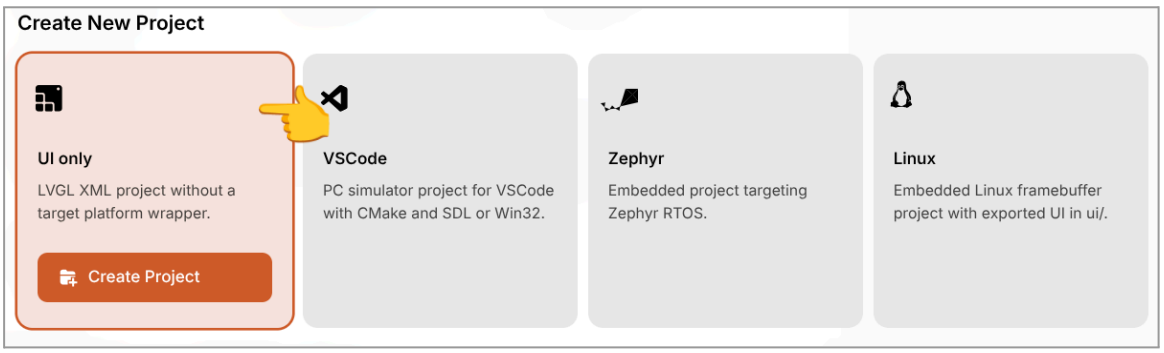


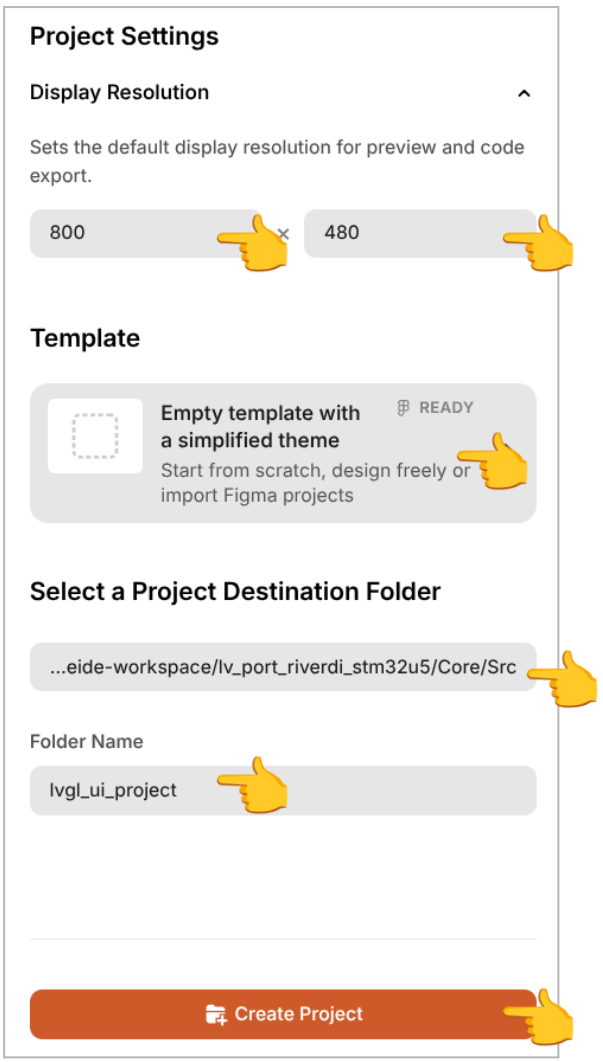
5. Compile it by clicking **Project / Build All**
6. **Launch LVGL Pro**, login by entering the OTP (One Time Password) and select the Community Licence
7. Login to **Figma**



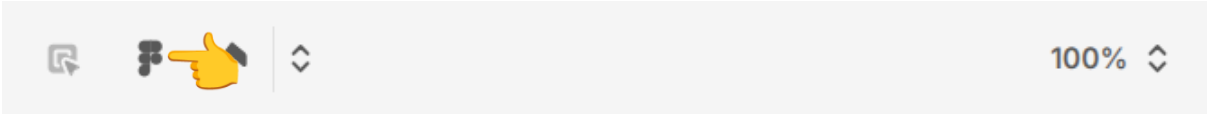
The Magic First - One click import from Figma

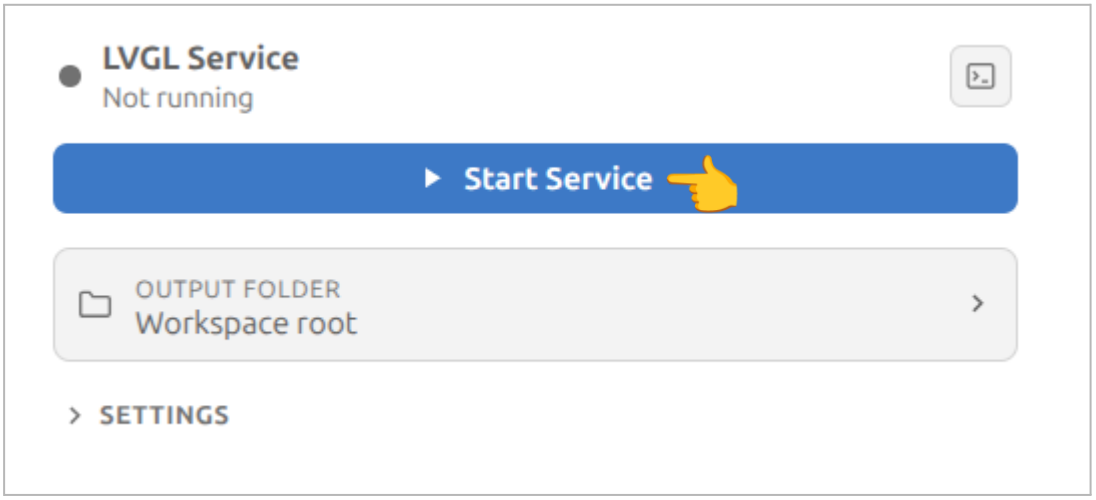
1. In LVGL Pro create a new project, set the resolution to 800x480 and select the Empty template use the UI in the CubeIDE project folder as a target





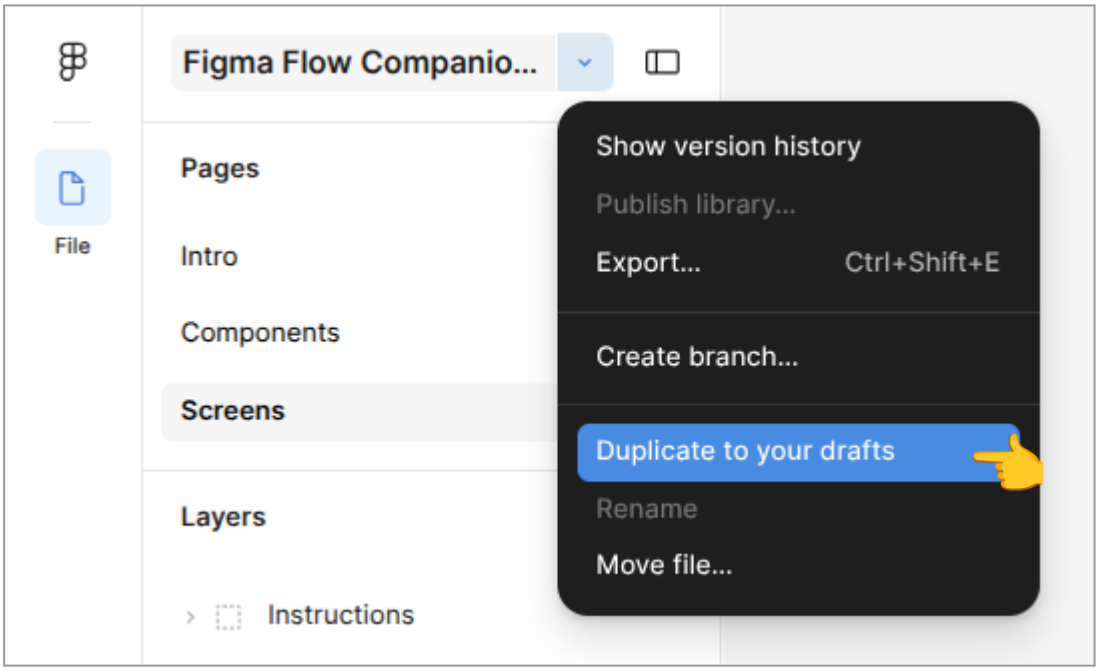
2. Start the **FigmaFlow Service** in the editor



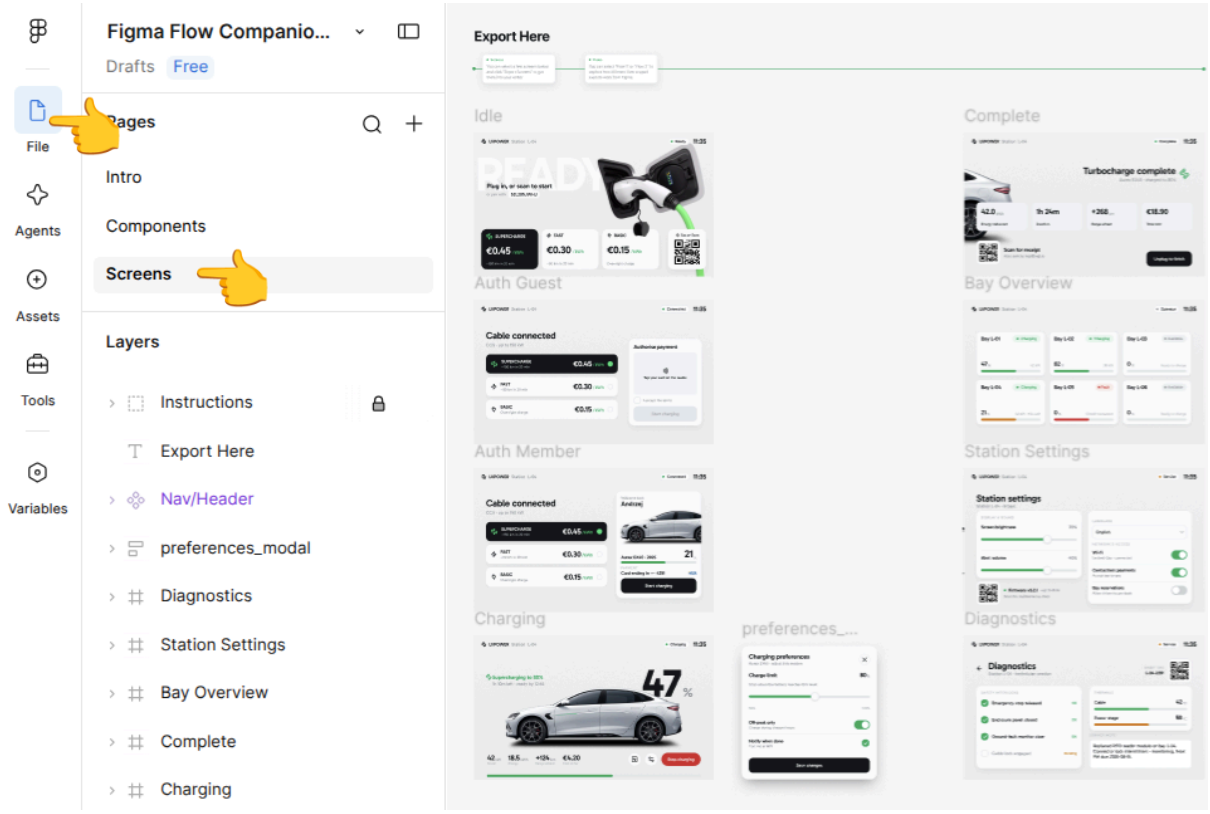


3. Go to this **Figma link** in **Google Chrome**: https://www.figma.com/design/VHJaDRALCGXncSI1udhctM/Figma-Flow-Companion-File?node-id=269-2&p=f

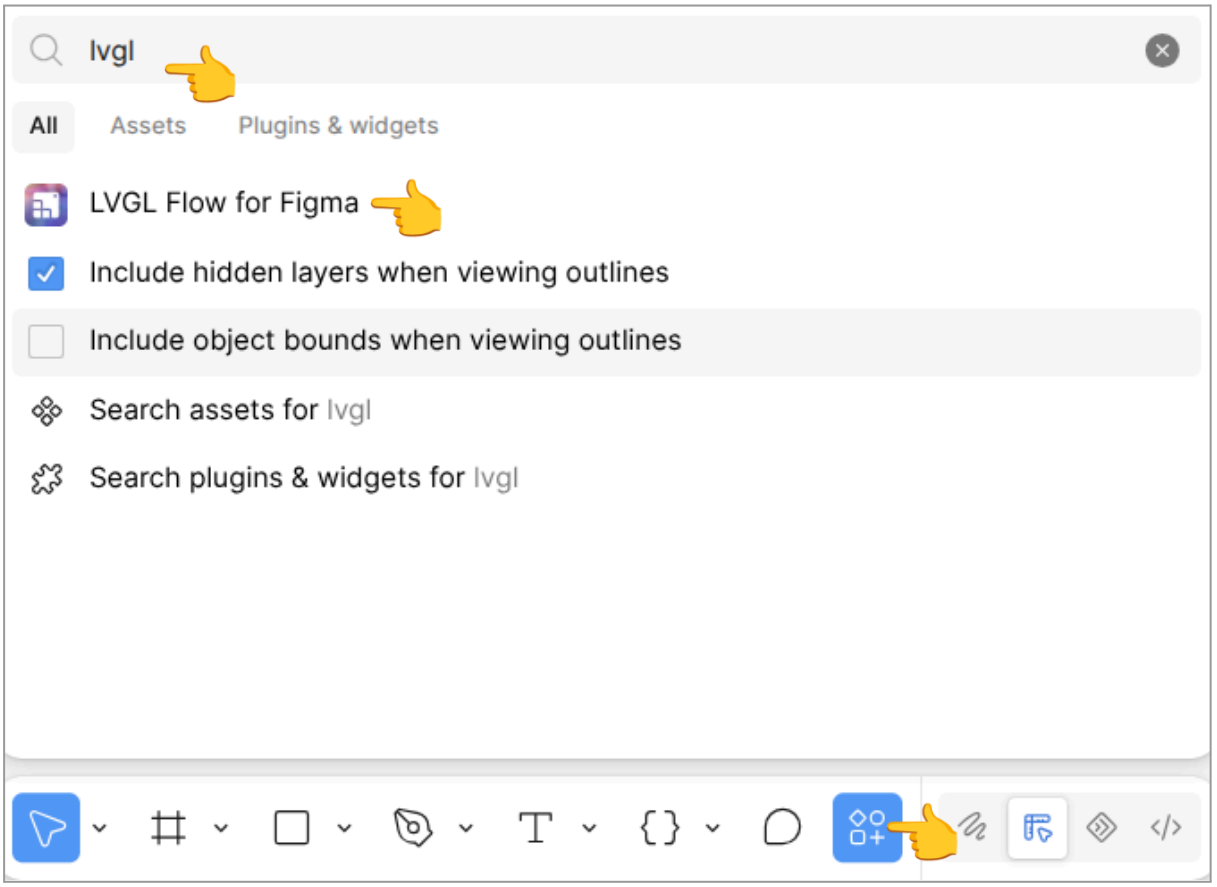
4. **Duplicate** the design (If Chrome blocked the popups, allow them and duplicate again)

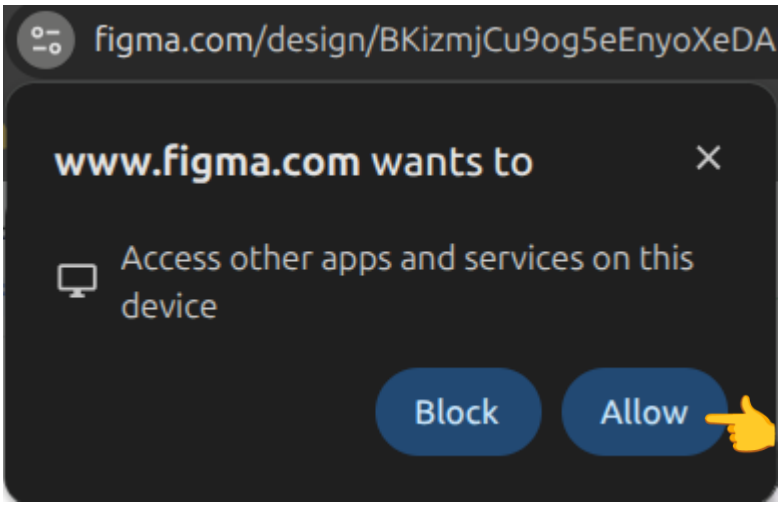


5. Open the design of the screens:

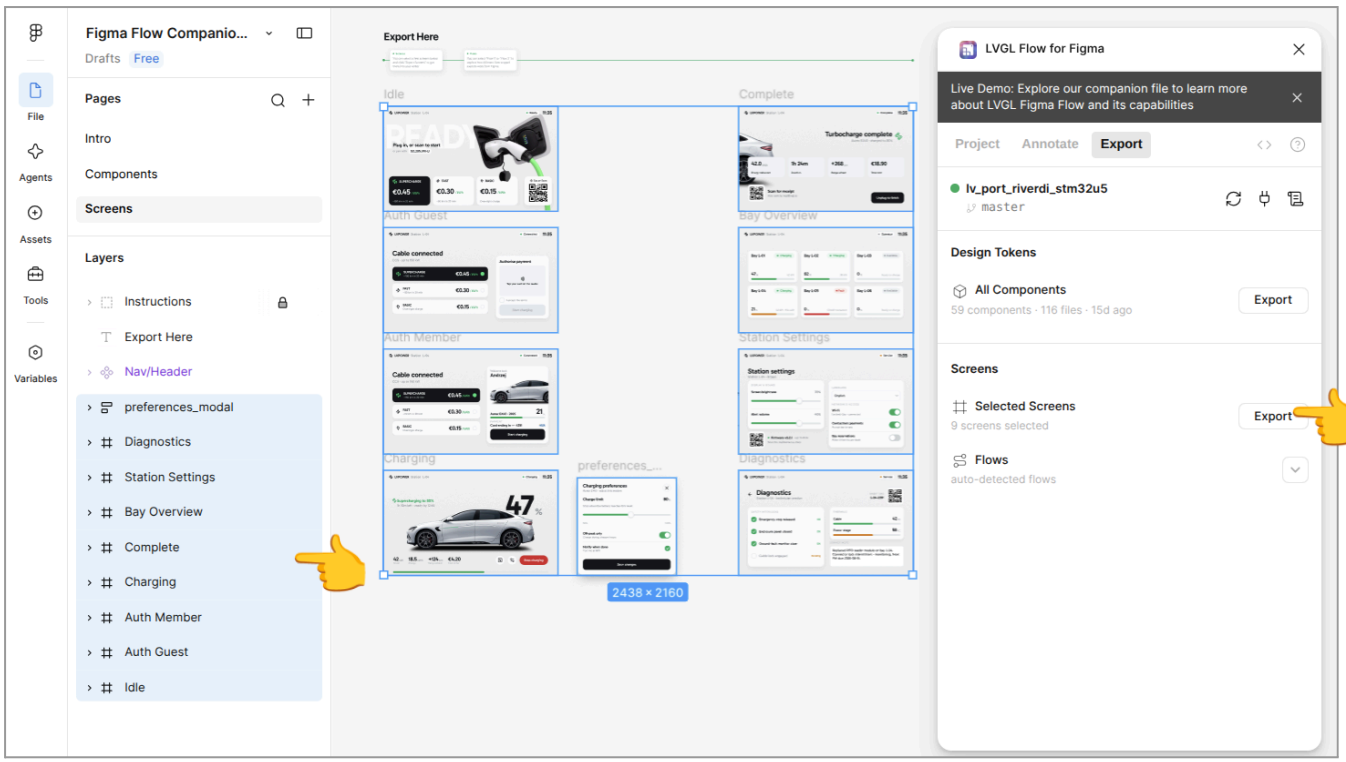


6. Start the **FigmaFlow plugin**

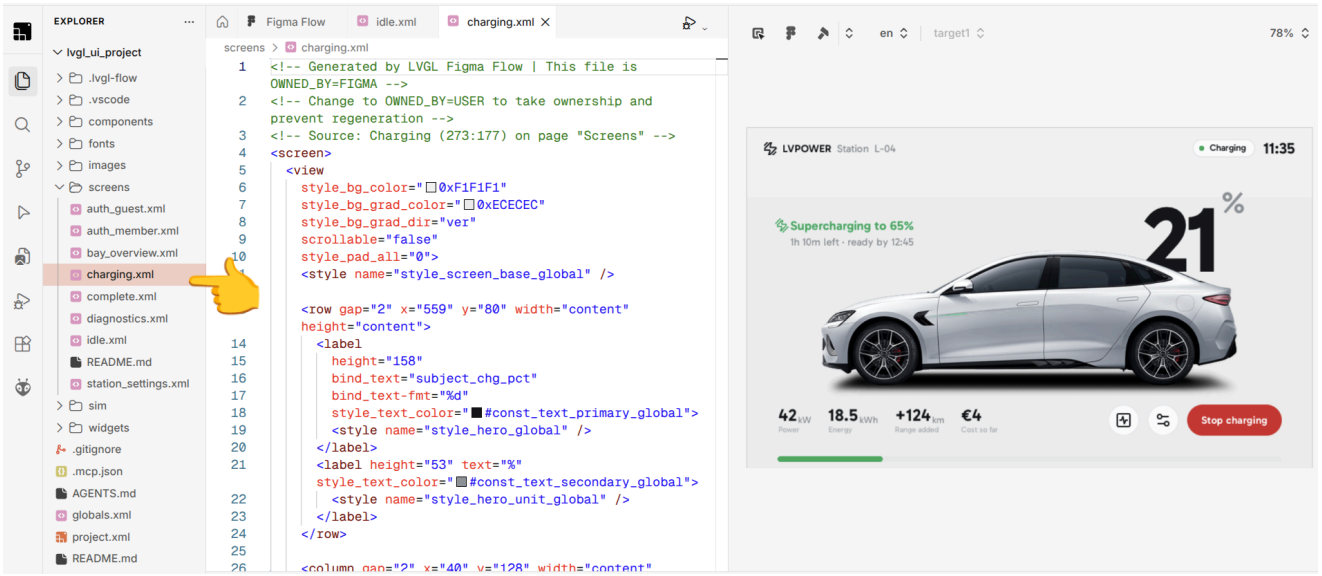




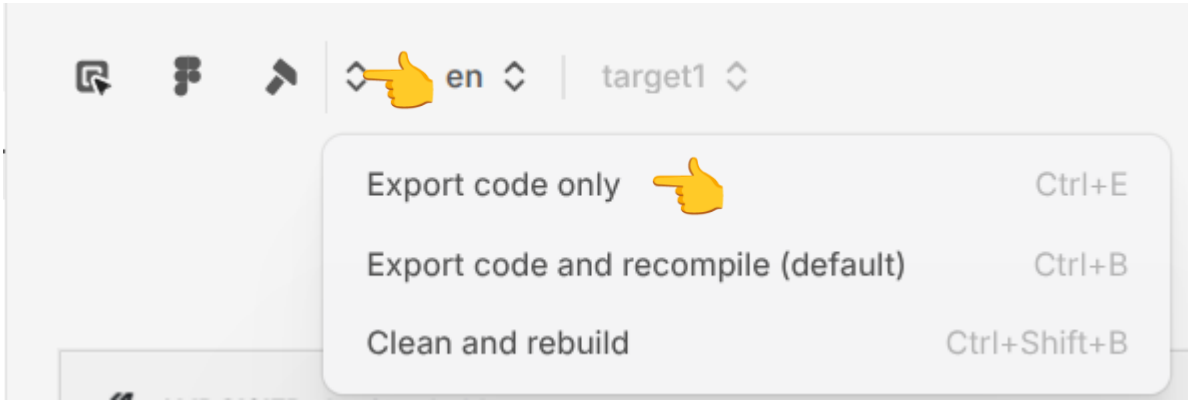
7. **Select all screens** and click **Export** next to the Selected Screens



8. Open **screens/chargin.xml** in **LVGL Pro** and try it out



9. **Export code** (Ctrl+E)



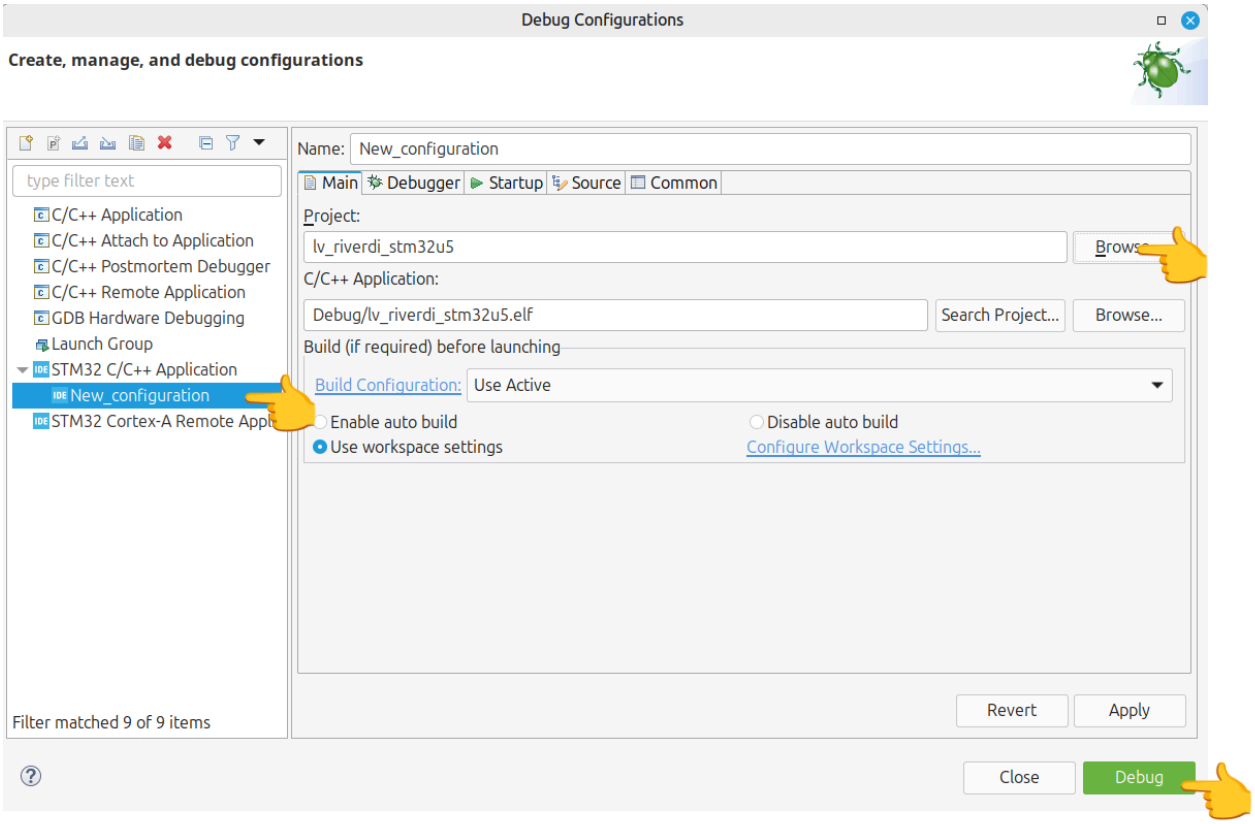
10. Open CubeIDE, import the project if not imported yet
11. Open **Core/Src/main.c** and at line 47 add

```
#include "lvgl_ui_project/lvgl_ui_project.h"
```

And at line 180 initialize the UI and load a screen

```
/* lvgl demo */
//lv_demo_widgets();
//lv_demo_music();
//lv_demo_benchmark();
lvgl_ui_project_init("");
lv_screen_load(charging_create());
```

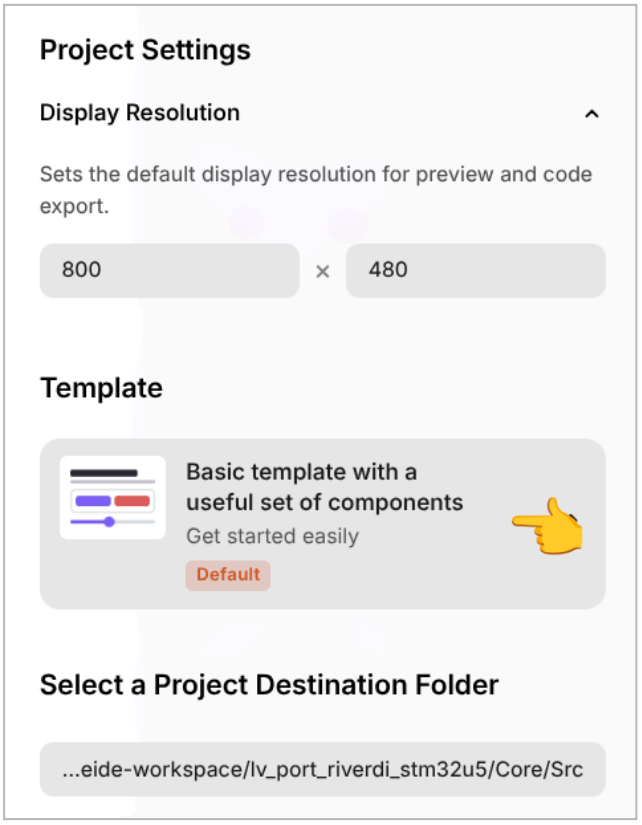
12. Connect the board.
13. Compile and flash. Create a new Debug configuration if needed.



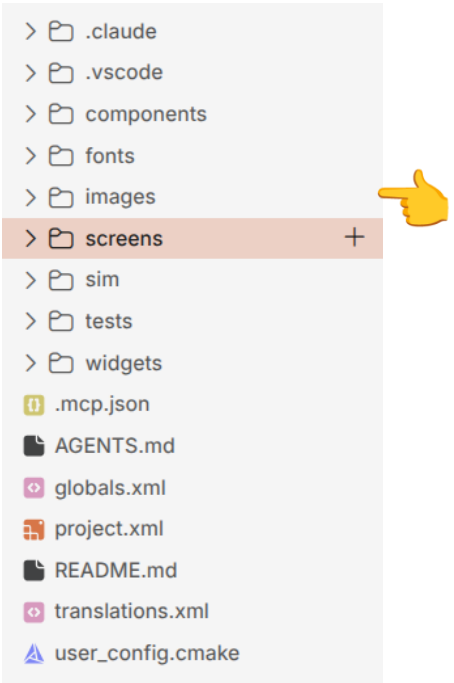
Learn the basics - Screen, components, styles, and data bindings

1. Move the `lvgl_ui_project` folder somewhere outside from the `Core/Src` CubeIDE project, so you can keep it.

2. Create a new project in LVGL Pro and use the **Basic template**



3. Create a **new screen** called **`screen_1`**. Hover over Screens and click the + button



4. Create a Hello world label and change a few properties:

```
<screen>
   <view>
       <lv_label
           text="Hello world"
           align="center"
```

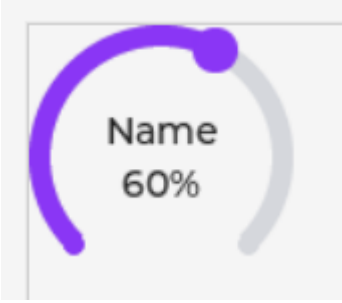
```
            style_text_font="font_h2"
            style_text_color="0xff0000"
        />
    </view>
</screen>
```

5. Create an `arc` as well and connect it to `subject_brightness`

```
<screen>
    <view>
        <lv_label
            text="Hello world"
            align="center"
            style_text_font="font_h2"
            style_text_color="0xff0000"
        />
        <arc subject="subject_brightness"/>
    </view>
</screen>
```

6. Add 2 labels to the arc. One with a simple text, and the other is bound to `subject_brightness`

```
        <arc subject="subject_brightness">
            <lv_label align="center" y="-10" text="Name" />
            <lv_label align="center" y="10" text="{subject_brightness .
'%'}" />
        </arc>
```



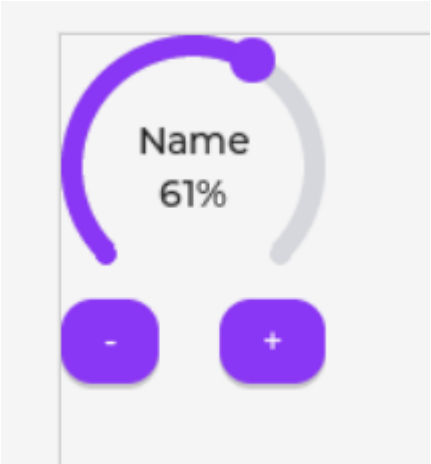
7. Add 2 buttons after `</arc>` to increment and decrement the subject.

```
        ...
        </arc>

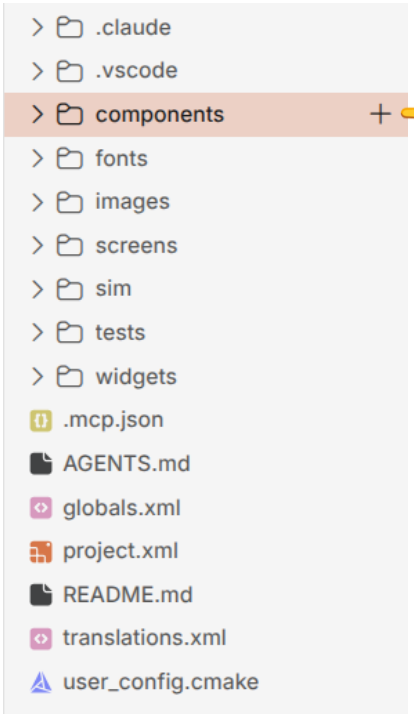
        <button text="-" x="0" y="100">
            <set_subject_event
                subject="subject_brightness"
                value="{subject_brightness - 1}"
```

```
            trigger="clicked"
        />
    </button>

    <button text="+" x="60" y="100">
        <set_subject_event
            subject="subject_brightness"
            value="{subject_brightness + 1}"
            trigger="clicked"
        />
    </button>
```



8. Create a **new component** called **`my_arc`** and copy the arc and the buttons to its `<view>`. Also add `width="content" height="content"` to the <view>.




```
    <view extends="lv_obj" width="content" height="content">

        <arc subject="subject_brightness">
            <lv_label align="center" y="-10" text="Name" />
            <lv_label align="center" y="10" text="{subject_brightness .
'%'}" />
        </arc>
        <button text="-" x="0" y="100">
            <set_subject_event
                subject="subject_brightness"
                value="{subject_brightness - 1}"
                trigger="clicked"
            />
        </button>

        <button text="+" x="60" y="100">
            <set_subject_event
                subject="subject_brightness"
                value="{subject_brightness + 1}"
                trigger="clicked"
            />
        </button>
    </view>
```

9. Add a <style> too

```
<styles>
        <style
            name="style_base"
            bg_color="0xffffff"
            pad_all="{space_md}"
            radius="{space_md}"
        />
    </styles>

    <view extends="lv_obj" width="content" height="content">
        <style name="style_base" />
```

10. Create 2 properties inside <api>: “room_name” with “string” type and “data” with “subject” type

```
    <api>
        <prop
            name="room_name"
            type="string"
            default="Room"
            help="Name shown under the arc"
        />
        <prop
            name="data"
            type="subject"
            default="subject_brightness"
            help="Int subject the arc shows and changes"
        />
    </api>
```

11. Route the properties to widget properties

```
    <view extends="lv_obj" width="content" height="content">
        <style name="style_base" />

        <arc subject="{data}">
            <lv_label align="center" y="-10" text="{room_name}" />
            <lv_label align="center" y="10" text="{data . '%'}" />
        </arc>
        <button text="-" x="0" y="100">
            <set_subject_event
                subject="{data}"
                value="{data - 1}"
                trigger="clicked"
            />
```

```
        </button>

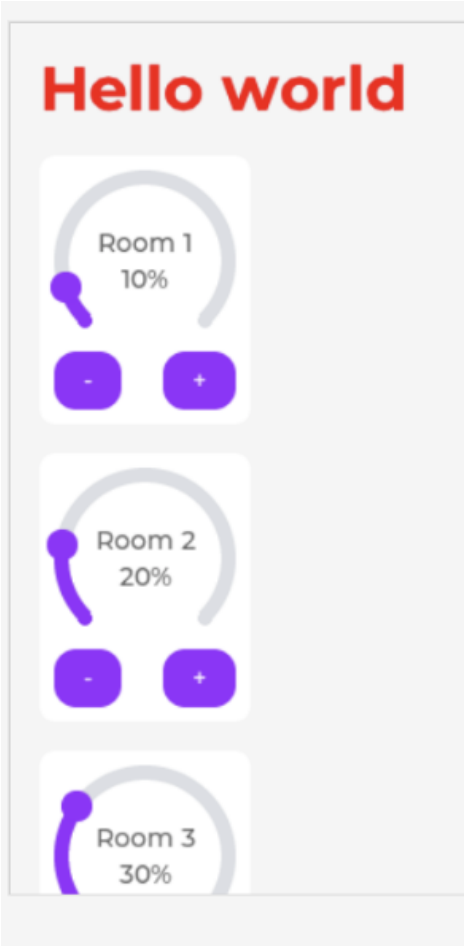
        <button text="+" x="60" y="100">
            <set_subject_event
                subject="{data}"
                value="{data + 1}"
                trigger="clicked"
            />
        </button>
    </view>
```

12. Create 3 subjects in globals.xml:

```
    <subjects>
        <int name="subject_room1" value="10"/>
        <int name="subject_room2" value="20"/>
        <int name="subject_room3" value="30"/>
        ...
```

13. On `screen_1` set a column layout and create 3 `my_arc` components

```
<screen>
    <view
        flex_flow="column"
        style_pad_row="{space_lg}"
        style_pad_all="{space_lg}"
    >
        <lv_label
            text="Hello world"
            align="center"
            style_text_font="font_h2"
            style_text_color="0xff0000"
        />
        <my_arc room_name="Room 1" data="subject_room1" />
        <my_arc room_name="Room 2" data="subject_room2" />
        <my_arc room_name="Room 3" data="subject_room3" />
    </view>
</screen>
```



14. **Export** the code (Ctrl+E).
15. Disable the widgets demo and benchmark demo in `Middlewares/Third_Party/LVGL/lv_conf.h`

```
#define LV_USE_DEMO_BENCHMARK 0
```

16. In CubeIDE, open `Core/Src/main.c` and replace `charging_create()` with `screen_1_create()`:

```
/* lvgl demo */
//  lv_demo_widgets();
//lv_demo_music();
//lv_demo_benchmark();
lvgl_ui_project_init("");
lv_screen_load(screen_1_create());
```

17. Compile and flash. Turn the arcs or press - and + to change the values on the board.